PART 5

Switching and route history

How the add-on remembers which way a train went. First the current keyframe-and-marker system and its limits, then a simpler model borrowed from real railways: **set the points, and the train follows its own occupied path**.

5.1 How switching works today
5.2 Live switching triggers
5.3 Remove Switch and auto-switch
5.4 Where the current model strains
5.5 Proposal: turnout state + occupied path
5.6 Scenarios walked through
5.7 Mapping today's UI onto it
5.8 Migration path

5.1 How switching works today

CURRENT STATE

Every switch keys **offset_factor and influence on every held constraint** at that frame. The newly active curve's influence key is set to 1 with CONSTANT interpolation, and all others to 0. A timeline marker `Switch: {object} -> {curve}` is added. Manual switches are **EXTREME** keys, reconstructed ones **BREAKDOWN**, and plain position keys are normal **KEYFRAME**s.

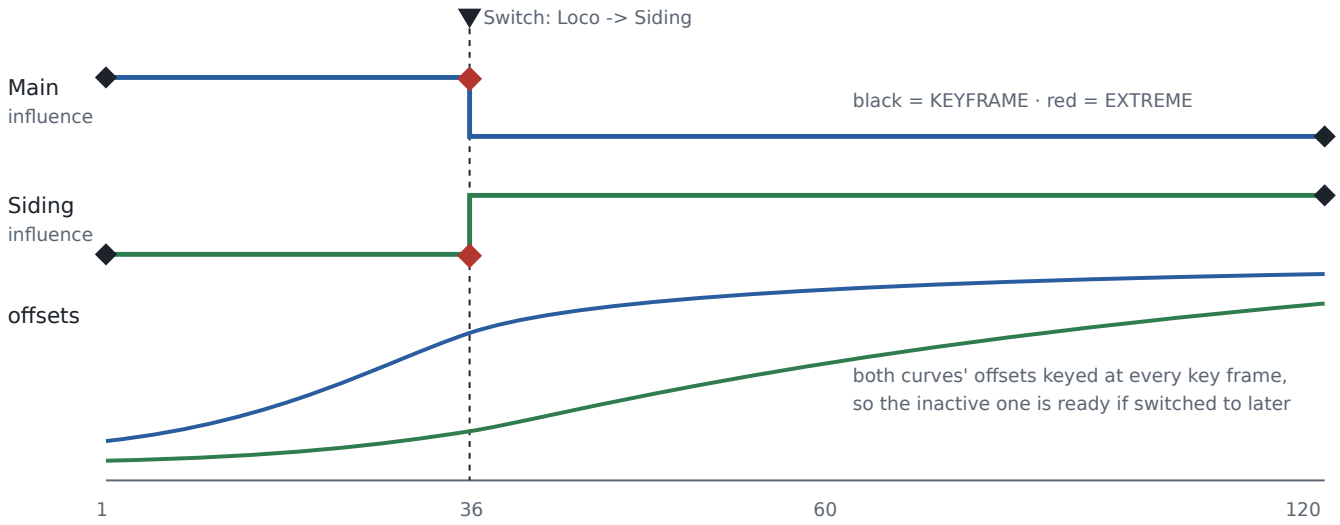


Figure 5.1 A single switch at frame 36, as the add-on records it now.

5.2 Live switching triggers

While dragging (with the Live Switching box ticked), the vehicle changes curve without writing keys, but **only** in two situations. That rule was written after two dead ends (switching at curve ends only, and switching to whatever was nearest):

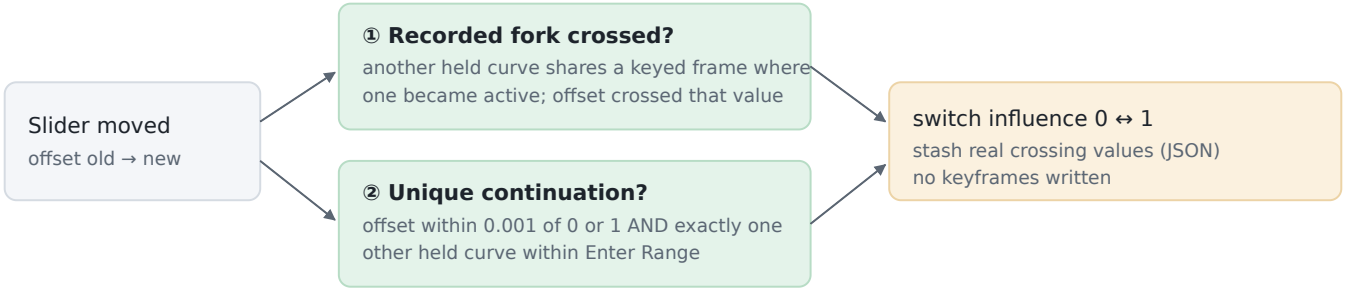


Figure 5.2 `_track_it_try_live_switch_continue`. A dead-zone (1e-4) stops an endless ping-pong at the threshold.

5.3 Remove Switch and auto-switch

Remove Switch (cascade)

On a switch frame, the Keyframe button becomes **Remove Switch**. It deletes this vehicle's switch markers and **every** offset/influence key at and after that frame, hand-placed ones included, after a dialog showing the real counts. Reason: everything later was built on a decision that no longer exists.

Auto-switch reconstruction

If you key the vehicle on a different curve than the last real key recorded (because you live-dragged across a fork), a **BREAKDOWN** switch key is inserted at the midpoint frame. It uses the **stashed real crossing offsets**, not the current resting position, so playback runs smoothly with no pop.

5.4 Where the current model strains

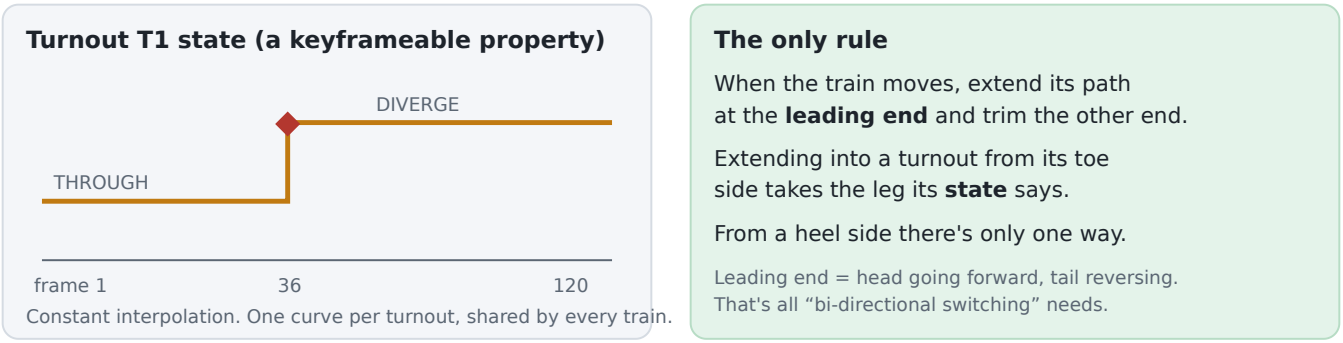
Situation	What happens now
Leftover test keys on a constraint	Keyframe mining reads them as real forks. This has happened twice (v8 §8.3).
Pivots or a trailing car reach a fork before the root	Pivots clamp to the root's curve. Wagons use a separate odometer log with dedupe, “authoritative” overwrite and stale-entry pruning.
Deleting an early switch	Must also delete every later key, because later offsets depend on it.
Two curves meeting at two junctions (loops, reversing)	The 1-D odometer maps two places to one distance. Pruning drops the older claim, which works but needs care.
Switchback (main → curve 2, reverse onto curve 3)	Handled by playhead position. It only works if keys are placed in the right order.
Two trains through the same turnout	Each vehicle mines its own keys, so nothing ties them to one real turnout.

Situation	What happens now
A detour inside a single frame	Can't be reconstructed at all (v8 §8.4).

None of these are coding mistakes. They all come from one choice: **storing the route as per-curve offsets on each vehicle**. Change what is stored, and they go away.

5.5 Proposal: turnout state + occupied path

On a real railway, trains don't choose. **The points are set, and the train goes where the rails lead.** The model below copies that:



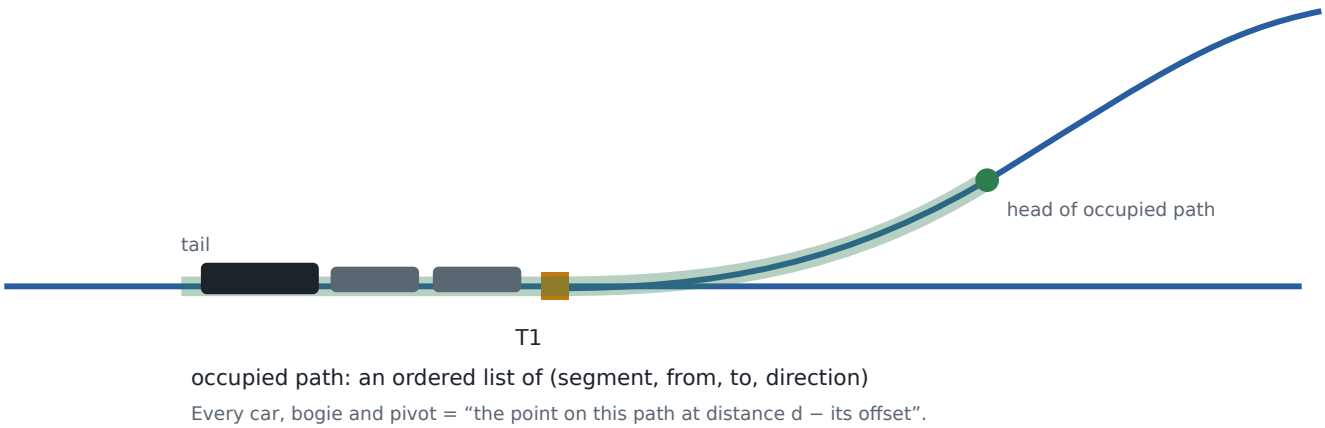


Figure 5.3 The train owns a contiguous **occupied path**. Points ahead of the train are decided only when the leading end reaches them; cars already on a leg stay on it.

Why this is deterministic when scrubbing

The occupied path at frame *F* comes from **replaying** the train's distance curve from its start frame to *F*, extending and trimming as it goes, and reading each turnout's state at the frame the leading end reaches it. That is a small step per frame, **cached per frame** and invalidated only when the distance keys, the turnout keys or the track change. Scrubbing backwards just reads the cache. Rendering frame 200 on a farm replays 1→200 once (milliseconds for normal scenes).

WHAT DISAPPEARS

- No per-curve offset keys on vehicles (offsets become outputs, bakeable).
- No keyframe mining, no shadow-tracking of held curves, no odometer drift or pruning heuristics.
- No cascade delete: removing a turnout key only changes where the train goes *afterwards*. Nothing downstream is invalidated, because distance is independent of route.
- No pivot/wagon special cases: they are all points on the same path.
- No overlap-length rule for consists (Part 4 §4.4).

Interlocking: safety for free

Changing a turnout while part of the train is already over it does nothing to the cars that are already past the toe, because they are on the occupied path. The UI can warn “T1 is occupied” and show the state change taking effect for the *next* train, just like real signalling. This is the gating rule from v8 §8.1 (“if any point has passed the divergence, the fork is no longer a choice”), and it happens automatically.

5.6 Scenarios walked through

Scenario	With turnout state + occupied path
Simple diverge at frame 36	Key T1 = DIVERGE at or before the frame the head reaches it. Key the train's distance as usual. Pivots and wagons each take the branch when *they* reach the toe.
Switchback: main → curve 2, then reverse onto curve 3	T1 (main/curve 2) = DIVERGE. The train runs onto curve 2 past T2. Key T2 = DIVERGE toward curve 3, then key the distance going **down**. The tail is now the leading end, it reaches T2 from its toe side, and takes curve 3. No special case.
Back a train into a siding, decided by the last car	The same thing: when reversing, the last car's end of the path extends through the siding turnout's state. The locomotive follows later on the same path.
Drag live without keys	Dragging changes distance and extends the path using turnout states at the *current frame*. Option: **“Drag uses: current state / next keyed state”**. The second gives the behaviour asked for in session 5 (“if a switch exists, dragging from before frame 36 still takes it”).
Two trains, one turnout	Both read the same T1 F-curve. Set it to DIVERGE for train A at frame 40, back to THROUGH at frame 90 for train B. That is a realistic yard timetable.
Remove a decision	Delete the T1 key. The train now stays on main. Its speed and timing keys still make sense, so nothing else is deleted.
Joint	Always a single way on, so the path just continues (unless the joint's auto-continue is off: buffer stop).

DECISION NEEDED

Per-train overrides? Some users will want “this train always takes the branch at T1, whatever the turnout says” (for example, crowd scenes). Recommendation: add an optional **Route Plan override** list per consist, an ordered list of (junction, leg) choices tried before the turnout state. It is off by default so there is one source of truth.

5.7 Mapping today's UI onto it

Today	Becomes
Switching panel: “Ready to Switch → X”	“Junctions ahead”: the next N junctions within look-ahead distance of the leading end, each with a THROUGH / DIVERGE toggle and a key button. The HUD “Switch Available” notice fires when the leading end is within range of an unset turnout.
Switch button (EXTREME key + marker)	Keys the turnout state (EXTREME type kept for visibility). The marker becomes `T1 → Siding` by junction uid, so renames are safe.
Keyframe Offset Factor	**Key Position**: keys consist distance. The label logic stays the same (key or remove).
Remove Switch cascade dialog	Plain “Remove turnout key”. The dialog can list which trains' routes will change.
Live Switching checkbox	Not needed. Live dragging always follows the network.
Enter/Exit Range	Only used by Scan Track (tolerances) and the HUD look-ahead.

5.8 Migration path

1. **Build the Track Network (Part 4)**
   It is useful on its own: it speeds up today's candidate discovery and fixes the yard false positive.

2. **Add Junction objects + state, alongside the old system**
   The Switch button writes both the old keys and the turnout key, so nothing breaks.

3. **Implement the occupied-path evaluator behind a toggle**
   Apply it to consists first, where the gains are biggest, then to pivots.

4. **Convert existing scenes**
   A **Convert Keyed Switches** operator reads each vehicle's EXTREME/BREAKDOWN keys and markers, creates turnout keys, and converts offset keys to a distance curve (integrating length along the resolved route).

5 **Retire the old path**
Remove keyframe mining, shadow tracking, stash JSON and the odometer log once the converter has been checked on the test scenes.
